# Android Car




**By**

**Saddam Hussain**
**2011-Umdb-11193**

**Haroon-ur-Rashid**
**2011-Umdb-11205**

**Junaid Ahmed Butt**
**2011-Umdb-11186**

**Supervisor**

**Engr. Muhammad Zaman Fakhar**
**Lecturer**

**DEPARTMENT OF SOFTWARE ENGINEERING**
**FACULTY OF SCIENCES& ENGINEERING**
**UNIVERSITY OF AZAD JAMMU & KASHMIR**
**MUZAFFARABAD**
**Session: 2011-2015**

## Special Dedication:

*To our beloved family for their genuine love, prayers and encouragement, then to our supervisor Mr. Muhammad Zaman Fakhar who guide and give us moral support and last but not the least to all our friends for their support and help throughout Our journey of education.*


بسم الله الرحمن الرحيم

# Android Car

By

Saddam Hussain

2011-Umdb-11193

Haroon-ur-Rashid

2011-Umb-11205

Junaid Ahmad Butt

2011-Umb-11186

Session

2011-2015

A thesis/project submitted in partial fulfillment of the requirement for the degree of

Bachelor of Software Engineering

Thesis/Project Supervisor:

Engr. Muhammad Zaman Fakhar

Lecturer Department of Software Engineering

External Examiner's Signature: _____

Thesis/Project Supervisor's Signature: _____

DEPARTMENT SOFTWARE ENGINEERING

FACULTY OF SCIENCES & ENGINEERING

UNIVERSITY OF AZAD JAMMU & KASHMIR,

MUZAFFARABAD

# ABSTRACT

Today's Remote control toys are controlled by a single dedicated controller. We endeavored to develop a remote control car that would be controlled via Bluetooth wireless technology. Bluetooth module enables you to transmit & receive wireless data in serial format. Additionally, Bluetooth consumes low power and has a larger bandwidth than the standard 900 MHz RF link which drives most toys. Here we can control the car through android application and also by PC using hyper terminal. Bluetooth module is connected to microcontroller serially with 9600 baud rate. The range of Bluetooth module is 15 meters with frequency 2.4 GHz. Android controlled Bluetooth car prototype showed the feasibility and practicality of Bluetooth controlled devices, while also revealing limitations in the amount of data transmission capable in commercially available Bluetooth options. In our project Bluetooth module acts as receiver and an android application acts as transmitter which is paired to the Bluetooth module. In this way we can control the car.

# UNDERTAKING

I certify that research work titled "*Android car*" is our own work. The work has not been presented elsewhere for assessment. Where material has been used from other sources it has been properly acknowledged/ referred.

Signature of Student

Saddam Hussain

2011-Umdb-11193

Signature of Student

Haroon-Ur- Rashid

2011 -Umdb-11205

Signature of Student

Junaid Ahmad Butt

2011 -Umdb-11183

# ACKNOWLEDGEMENTS

Praised be to Allah for his blessings and giving us the strength for completing this report. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend our sincere thanks to all of them.

We are highly indebted to **Engr. Muhammad Zaman Fakhar** for their guidance and constant supervision as well as for providing necessary information regarding the project and also for their support in completing the project.

I would like to express my gratitude towards my parents and friends for their kind co-operation and encouragement which help us in completion of this project.

Our thanks and appreciations also go to our colleague in developing the project and people who have willingly helped us out with their abilities.

# TABLE OF CONTENTS

## Table of Contents


***ABSTRACT*** ..... ***3***

***UnderTaking*** ..... ***4***

***Acknowledgements*** ..... ***5***

***Table of Contents*** ..... ***6***

***Table of Contents*** ..... ***7***

***List of figures*** ..... ***8***

***List of tables*** ..... ***9***

***Chapter 1*** ..... ***10***

1.1 Introduction……10

1.2 Scope of Project ……11

1.3 Objectives……11

1.4 Existing Application Problem……12

1.5 Proposed System……12

1.6 Technology to be used……12

***Chapter 2*** ..... ***14***

2.1 Requirements ..... 14

2.2 Specific Requirement ..... 15

2.3 Software Requirement ..... 16

2.4 Hardware requirement ..... 16

2.5 Overall Description ..... 17

2.6 Security ..... 19

***Chapter 3*** ..... ***21***

3.1 Need for Android controlled car: ..... 21

3.2 Key features of the proposed system ..... 23

3.3 Mechanism Of Project: ..... 24

***Chapter 4*** ..... ***25***

4.1 Architecture: ..... 25

4.1.1 Client Application Architecture……25

4.2 Design Implementation: ..... 28

***Chapter 5*** ..... ***32***

5.1 Tools & Technologies Used in Software Development ..... 32

5.1.1 Eclipse……32

5.1.2 Android Software Development Kit……33

5.1.3 Blue Stack……34

5.1.4 Proton IDE……35

5.1.5 Proteus 7.8……36

*5.2* Abstract ..... ***37***

5.2.1 Coding Standards…… 37

5.2.2 Debugging ..... 38

5.2.3 Code walkthrough ..... 39
5.2.3.1 Code Walkthrough Objectives: ..... 39
5.2.3.2 Code walkthrough details ..... 40
5.2.4Code Inspection: ..... 42
5.2.4.1 Code Inspection Objectives: ..... 42
5.2.4.2 Code inspection details ..... 42
***Chapter 6*** ***..... 45***
6.1 Introduction: ..... 45
6.2 Scope of Testing ..... 45
6.3 Test Plan Strategy ..... 46
6.4 Requirements Verification and Validation ..... 48
6.5 Solution validation and Analysis of data ..... 49
6.6 Test Environment and testing Schedule ..... 50
6.7 Resources ..... 50
6.8 Test Cases Implementation ..... 52
***Chapter 7*** ***..... 59***
Interface Design ..... 59
***Conclusions & Recommendations*** ***..... 62***
Result ..... 62
Discussion ..... 62
Conclusions ..... 62
Recommendations ..... 63
**References ..... 64**
***Abbreviations*** ***..... 67***

# LIST OF FIGURES


- Figure 1: Block Diagram of the Whole Sys0tem
- Figure 2: Data flow Diagram of the whole system.
- Figure 3: Android BT Car
- Figure 4: Android BT Car
- Figure 4.1: Android Car (user interface)
- Figure 5: Architectural Diagram
- Figure 7: User interface of App
- Figure 8 : User interface of App
- Figure 9: Use case Diagram
- Figure 10: Sequence Diagram
- Figure 11:Class Diagram
- Figure 12:Circuit Diagram
- Figure 13: Eclipse User Interface
- Figure 14: Main Interface of Proton IDE
- Figure 15:Select Program to run on Proton IDE
- Figure 16: Eclipse Interface Showing Debugging
- Figure 17: Eclipse  Interface and Development Environment
- Figure 18: Application Control Panel
- Figure 19: Main Controlling Interface
- Figure 20: Help Interface
- Figure 21:About Interface

# LIST OF TABLES


- Table 5.1: System Requirements
- Table 5.2: Table 1 Code Walkthrough
- Table 5.3: Table 2 Code Walkthrough
- Table 5.4: Table 3 Code Walkthrough
- Table 5.5: Table 4 Code Walkthrough
- Table 5.6: Table 1 Code Inspection
- Table 5.7: Table 2 Code Inspection
- Table 5.7: Table 3 Code Inspection
- Table 5.8: Table 4 Code Inspection
- Table 5.8: Table 5 Code Inspection
- Table 6.1: Testing Schedule
- Table 6.2: White Box Testing
- Table6.3: Black Box testing
- Table 6.4: Unit Testing 1
- Table 6.5: Unit Testing 2
- Table 6.6: Unit Testing 3
- Table 6.7: Unit Testing 4
- Table 6.8: Unit Testing 5

# CHAPTER 1

# INTRODUCTION

## 1.1 INTRODUCTION:

In the last decade Smart Phones have increased exponentially in not only popularity, but also technical design. Software of the integrated systems on these handheld devices is under constant development, but is often limited by the increasing demand for compact size. Many current technologies exist that enable smart-phones to use a Bluetooth connection to control end device.

However, using an Android based smart-phone to wirelessly control the Android car the smart-phone must be able to wirelessly send commands for the movement and direction. These wireless capabilities require the use of some form of pre-created network.

Usually Robotic car is controlled through a remote using an RF link which is subject to attenuation, noise and has a very limited range as well.

This project features an Android Car that uses HC-06 standard which send AT commands to the device to make possible continuous connection. Bluetooth support high data rates which enables good quality for transmission from the Android Car to the smart-phone. Android Car can be controlled through any smart-phone wirelessly through a Bluetooth module using smart-phone.

Controlling a smart-phone by hand to control the car, forward key is map to move car in forward direction while reverse key is map to move in opposite direction. Turning left and rights to a car left and right keys are use.

This Android Car is capable of moving up to a distance of 15m with precise speed from the controlling smart-phone. It has rechargeable batteries which can support 1 hours of full operation. It is applicable for flat areas and also be applicable for

like blocked tunnels, pipe lines and enemy zones which are not reachable due to high risk level, blockage or because of our physical limitations.

In the ever changing, ever progressing age that we live in today, more and more tasks back then handled by humans are now replaced with robotic car. Though some may think those robotic cars are slowly becoming a threat to humans especially when it comes to jobs, some robotic car is designed to ease a person's life. The robotic car commonly used to defuse a bomb, enter into rubble could save a humans life which is more important compared to a mere job. Therefore, this project was chosen.

This project is just the beginning to multiple future advancement of a remote control car. This Android Car will be able to discover distant terrains be it hazardous or not.

## 1.2 Scope of application

This project is to develop a remote control car using Bluetooth communication and android devices that will enable control via Smartphone. The Android Car has a limited forward, backward, left and right movement. The speed and direction is not fixed and not potentiometer. This Android Car is high-end car which use to give precise speed and direction control. This project is developing a smart-phone by using Java programming language. To designing interfaces Bluetooth module with android phone have to take the Android Car application as a client server socket and lastly, perform system testing through the evaluation of hardware and software.

## 1.3 Objectives:

There are three main objectives in pursuing the accomplishment of this project:

- To study on smart-phone application using android: Respond to navigation commands triggered through a smart-phone using Android touchscreen smart phone in performing

- To control Android Car using Bluetooth communication: Maintain continuous wireless connectivity of smartphone

- To create a unified technical solution: To design robotic car that guide user using an Android Car in indoor environment or specific location.

## 1.4 Existing Application Problem:

- Previous mobile car used Radio Frequency (RF) that had problem on transmission frequency, such as low data rate and the frequency sometimes can be shared to other frequency devices.

- Utilize a commercially available smart phone (for example an Android phone) to command and control the Android Car. These capabilities are applicable to multiple scenarios including military missions, police surveillance and search and rescue activities. The prototype should be developed and demonstrated using commercially available products.

- The range of Android Car has limits.

## 1.5 Proposed System:

The Android controlled car is a project which facilitates users to control their robotic car from a distance with optimum mobility. All that is required to have remote access to your hardware device is an android phone. The Bluetooth pass key is the only thing user must know and by entering the correct pass key user will be able to remotely access his robotic car. We provide this communication between systems (Robotic car, Android Phone) via Bluetooth which is a very common communication medium these days.

Even at very bad weather no effect of attenuation and with a baud rate of 9600 bps this application runs like a charm and a very simple interface allows all kind of users to use it easily.

## 1.6 Technology to be used:

- Android Phone 4.2 or above
- Windows 10 Operating System
- PIC microcontroller
- PCB board
- L293D Motor controller

- 5V regulator
- HC-06 Bluetooth module
- 9V DC motors
- 9V Batteries
- LED lights
- Borland
- Eclipse Studio/Android Studio
- Java Platform JDK
- Android SDK
- Proton IDE
- Proteus 7.8
- Hyper Terminal
- Adobe Photoshop CS6
- Microsoft Visio 2013

CHAPTER 2

# Requirement Analysis

This chapter focuses in detail the need for building Android Application Software. The requirement analysis section of this document focuses on some of the key needs of why there was a need to develop such an Application. Requirement analysis basically process of understanding what the need of such Software was and once it will be developed what problem it will solve.

In the Era of technology each and every person is using Mobile Phone. In current situation of Pakistan and around the world there is a threat using mobile phone for communication. People use them to take photographs, send text, update Facebook and search the web.

## 2.1 Requirements

### 2.1.1 Android Application Requirements and Description

As mentioned in the introduction of this thesis, an Android application acts as a remote control device for a Bluetooth connected robotic car. For this project the Android device acts as a client to the robotic car. The touch screen feature of the Android device is used to control the car in different direction. All the information from the phone needs to be sent to the hardware device. For the exchange of information between the Android application and hardware device, Bluetooth is used as a mediator. The functions such as enabling the Bluetooth in a device, discovering Bluetooth car, creating a connection to the hardware device and directions are all done in the user side. Figure 1 shows the whole actions performed by the Android Bluetooth control car.

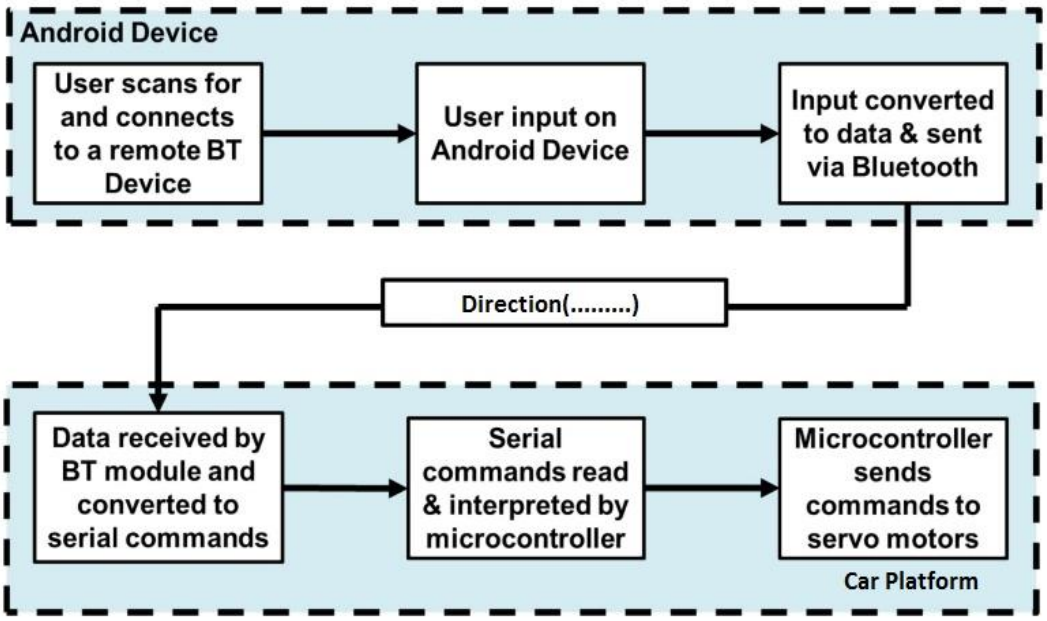



Figure 1: Block diagram of Whole System

## 2.2 Specific Requirement

This section of the chapter focuses on the Functional, Hardware and Software requirements of the Android Phone Application. All core, Major and Supporting requirement will be discussed in this section.

### 2.2.1 Functional Requirement

Functional requirements describe what the Android Mobile Application is expected intended for remotely accessing robotic car. Functional requirements of the system are:

- Pairing devices (pair both robotic car and application)
- Controlling hardware device (Remotely controlling)
- Directing car (moving car in all four direction)
- Exit (shutdown Bluetooth devices)

## 2.3 Software Requirement

This project requires the following software.

- Android Phone 4.2 or above
- Windows 10 Operating System
- Borland
- Eclipse Studio/Android Studio
- Java Platform JDK
- Android SDK
- Proton IDE
- Proteus 7.8
- Hyper Terminal
- Adobe Photoshop CS6
- Microsoft Visio 2013

## 2.4 Hardware requirement

This project requires the following hardware.

- Android Phone
- Laptop
- PIC Microcontroller
- PCB(Printed circuit board)
- Motor Driver L293D
- Resistor 10k
- Non polar capacitors 104pF
- Capacitors 10uF
- Crystal oscillator 4MHZ
- Voltage Regulator (L 78050v)  5V
- Diode 4007
- HC-06 Bluetooth module
- DC motors
- Heat sink

- Battery 12 Volt
- LED lights
- Switches
- Wheels
- Flexi Glass
- Jumper wire

## 2.5 Overall Description

### 2.5.1 Requirement Elevation

After a detailed analysis we reached at the conclusion that “Android Bluetooth Controlled Car” will serve our needs. We decided to use XML as a front end design and java as a backend for the development of client application. On the hardware side we used assembly language in embedded C burned in Microcontroller.

### 2.5.2 Economic Feasibility

Eclipse Studio, Blue stack, Android SDK, JDK, Proteus 7.8, Proton IDE, HyperTerminal are all available on the internet for free and are also open source which can be downloaded easily from the internet. So, the development of software is almost free, but Hardware components are not available for free so it makes some cost.

### 2.5.3 Technical Feasibility

Project is technically feasible because of the following reasons:

Java is the one of the most widely used programming language so, many software are built using java. It is somewhat a universal language for all platforms. We choose java for the development of Android application because there is lot of help and solutions for the problem which are encountered during the application development. Whereas C language is also one of the most widely used language and lot of work has been done in C language so that we use C language in developing hardware which makes easy to encountered problems during hardware development.

### 2.5.4 Legal Feasibility

The system is based on both hardware and software that does not concern to any organization that is the reason our system is legally feasible.

### 2.5.5 Requirement Specification

In this project Android application is use by the user to control robotic car. Application contains the user interface of the application. Most of the time during the use of application the user interacts with the Android application part than the hardware. All the works such as pairing, sending data and controlling robotic car wirelessly over Bluetooth are done by the user application. Figure 2 shows the System flow chart of the Android Bluetooth controlled car.

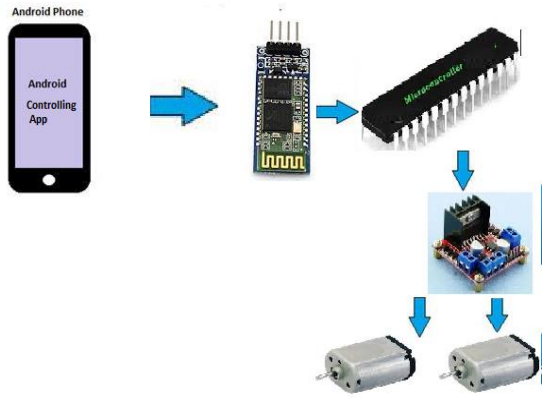



Figure 2: System flow Diagram of the Android BT controlled car.

Figure 2 shows the steps that take place during the use of the application to control robotic car. When the application is started the availability of Bluetooth is checked. Before using this application, it should be kept in mind that both Android application and hardware system should have Bluetooth connection. If available the application prompts to scan the Bluetooth device which we write the name of Bluetooth in the text box in application. If not already connected, the application provides a dialog box giving a message of socket error and also if robotic car is not connected Bluetooth blinks, when it connected its light become stable. The robotic car must already be active when the application sends the request for connection.

It should be noted that for making pair with robotic car application will require pin key for more secure connection, after accepting valid pin key robotic car will be able to build a connection with application. User can now able to control car and moves it in all four direction. The user can now simply end the connection by closing the application or switch off the robotic car. It should be noted that the connection will be built only if the robotic car is connected to same network.

### 2.5.6 Limitations:

At this point this application has a few limitations like:

- The devices will be connected only if they are on the same network.
- The range to which the user can access the robotic car is about 10-15m (<15).
- Bluetooth is a good communication system but not effective while working in a longer range.
- The rechargeable battery can give power up to one hour.
- Car cannot be move effectively in rough surface.

## 2.6 Security

Security is classified into different layers depending on the type of content intended to be secured.

### 2.6.1 Personal security:

The use of pass key makes the communication safe and secure between user and device.

### 2.6.2 Communication's security:

The use of AT protocol ensures the communication is done without loss and even at low battery power.

### 2.6.3 Network security:

The use of serial communication, sockets and ports makes the communication secure.

### 2.6.4 Information security:

The information exchange during the connection is safe and loss less.

# CHAPTER 3

# Need and Mechanism

This Chapter is mainly focused on the planning to establish a contextual basis for this thesis as a whole. This chapter starts off with brief introduction of Android controlled car and describes the design phase of application.

The objective of this project is to develop a remote control car using Bluetooth communication and android devices that will enable control via Smartphone. Android Car has a limited forward, backward, left and right movement.

## 3.1 Need for Android controlled car:

In the last decade Smart Phones have increased exponentially in not only popularity but also technical design. Software of the integrated systems on these handheld devices is under constant development but is often limited by the increasing demand for compact size. Many current technologies exists that enable smartphones to use a Bluetooth connection to control end device.

However, using an Android based smart-phone to wirelessly control the Android car the smartphone must be able to wirelessly send commands for the movement and direction. These wireless capabilities require the use of some form of pre-created network.

Usually Robotic car is controlled through a remote using an RF link which is subject to attenuation, noise and has a very limited range as well.

This project features an Android Car that uses HC-06 standard which send AT command to the device to make possible continuous connection. Bluetooth support high data rates which enables good quality for transmission from the Android Car to the smart-phone. Android Car can be controlled through any smart-phone wirelessly through a Bluetooth module using smartphone.

Controlling a smartphone by hand and finger motions alone is more intuitive than using both hands to control the smart-phone. To be able to control the car with one

hand, forward key is map to move car in forward direction while reverse key is map to move in opposite direction. Turning left and rights to a car left and right keys are use.

This Android Car is capable of moving up to a distance of 15m with precise speed from the controlling smartphone. It has rechargeable batteries which can support 1.5 hours of full operation. It is applicable for flat areas and also be applicable for like blocked tunnels, pipe lines and enemy zones which are not reachable due to high risk level, blockage or because of our physical limitations.

In the ever changing, ever progressing age that we live in today, more and more tasks back then handled by humans are now replaced with robotic car. Though some may think those robotic cars are slowly becoming a threat to humans especially when it comes to jobs, some robotic cars are designed to ease a person's life. The robotic car commonly used to defuse a bomb, enter into rubble could save a humans life which is more important compared to a mere job. Therefore, this project was chosen.

This project is just the beginning to multiple future advancement of a remote control car. This Android Car will be able to discover distant terrains be it hazardous or not.



Figure 3: Side view of Android Controlled Car

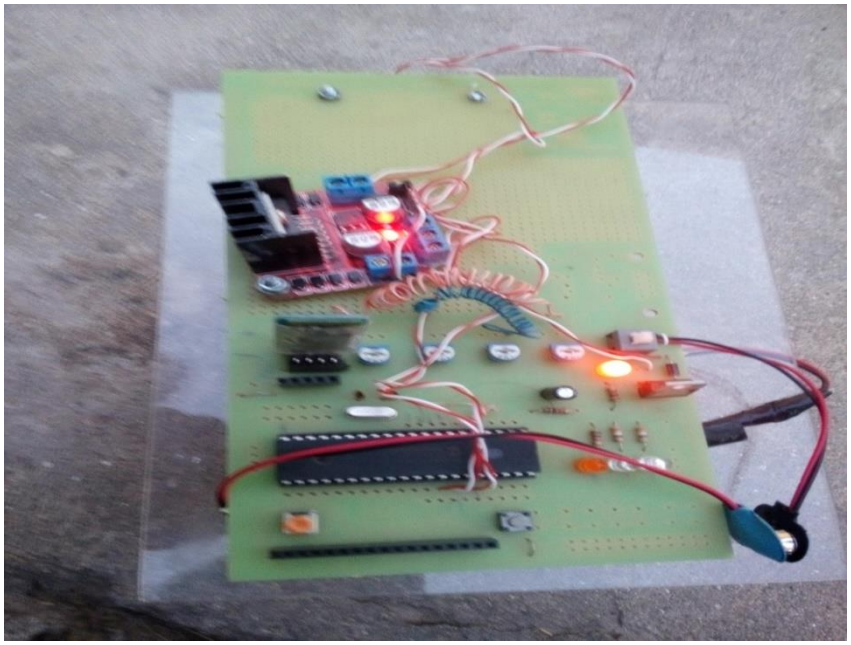


Figure 4: Top view of Android Controlled Car

## 3.2 Key features of the proposed system

The key features of the proposed system are as follows:

- The application is developed Android architecture considering its compatibility over different Android phone devices.
- The application is very carefully developed considering the limited available memory and battery life in smartphone and hardware device.
- The application has very simple design due to which it can be used by all kinds of users.
- The application gives user the facility to control robotic car over Bluetooth through his smart phone.
- Even at low communication speed the application runs smoothly without much delay and interruptions.

## 3.3 Mechanism of Project:

The application works as mentioned in the following manner:

- Firstly the user should open the application on his mobile.
- Then built your HC-06 Bluetooth module pair with your mobile.
- Password for pairing is “1234” or “0000”.
- Write name of the slave Bluetooth name in the text box and then tap connect button to pair Bluetooth module.
- When press “up arrow” it sends the data “A” to Bluetooth module connected with the circuit. When microcontroller detects “A” the robotic car moves FORWORD.
- When press “DOWN ARROW” it sends the data “B” to Bluetooth module connected with the circuit. When microcontroller detects “B” the robotic car moves REVERSE.
- When press “LEFT ARROW” it sends the data “C” to Bluetooth module connected with the circuit. When microcontroller defects “C” the robotic car turns LEFT.
- When press “RIGHT ARROW” it sends the data “D” to Bluetooth module connected with the circuit. When microcontroller defects “D” the robotic car turns RIGHT.
- The user can exit his application or he can switch off robotic car circuit.

# CHAPTER 4

# Architecture and Design

This chapter includes detailed information of the work during the design phase of the system which includes the design and architecture in detail. Best practice for designing for project is discussed in detail.

## 4.1 Architecture:

### 4.1.1 Client Application Architecture

Android application for this project acts as the client and car hardware is acts as a server. The client application contains the user interface of the application. Most of the time during the use of application the user interacts with the client part than the server. All the works such as connection to the hardware, sending data to the hardware device, controlling the robotic car wirelessly via Bluetooth are done by the client application. Figure 6 shows the architectural diagram of android controlled car application.

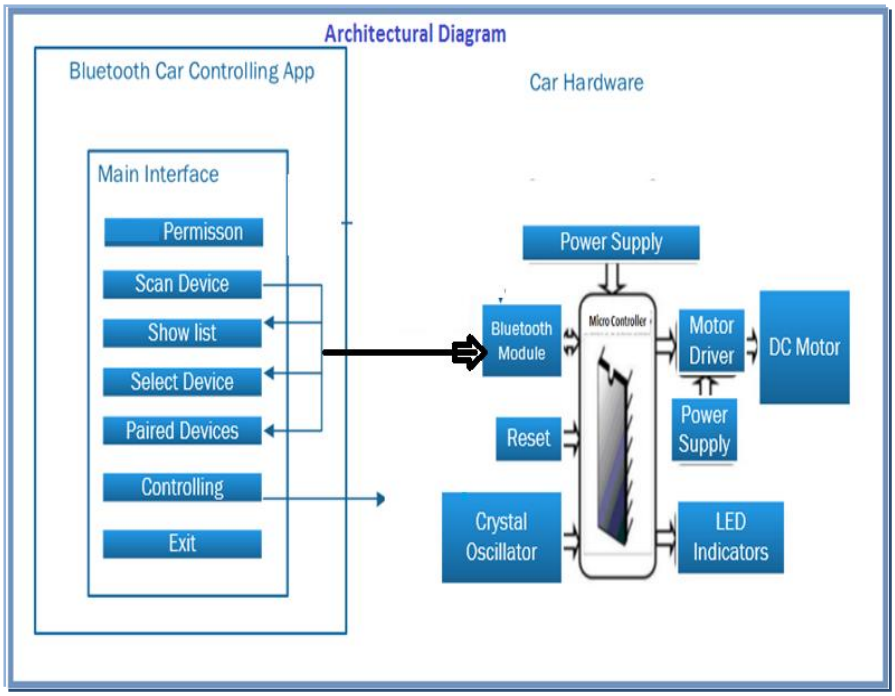



Figure 6: Architectural diagram of Android controlled car application

Figure 6 shows the steps that take place during the use of the application. When the application is started the availability of Bluetooth is checked. Before using this application, it should be kept in mind that both Android application and hardware system should have Bluetooth connection. Put the name of Bluetooth module in text box of the application then the application prompts to scan the Bluetooth device. If not already connected, the application provides a dialog box giving a message of socket error and also if robotic car is not connected Bluetooth blinks, when it connected its light become stable. The robotic car must already be active when the application sends the request for connection.

It should be noted that for making pair with robotic car application will require pass key for more secure connection, after accepting valid pass key robotic car will be able to build a connection with application. User can now able to control car and moves it in all four direction. The user can now simply end the connection by closing the application or switch off the robotic car. It should be noted that the connection will be built only if the robotic car is connected to same network.

The user interface in an Android application can be designed by coding in an xml file. Different types of layouts, buttons, colors and other user interface elements can be added to the xml file. Different other elements such as dialog boxes, text box, widgets, and images can also be added to the views. The xml provides the views that appear to a user. Figure 6 shows the user interface of the android application.




Figure 7: Android controlled car App (user interface)




Figure 8: Android interface with Bluetooth Connectivity

## 4.2 Design Implementation:

### 4.2.1 Use Case Diagram

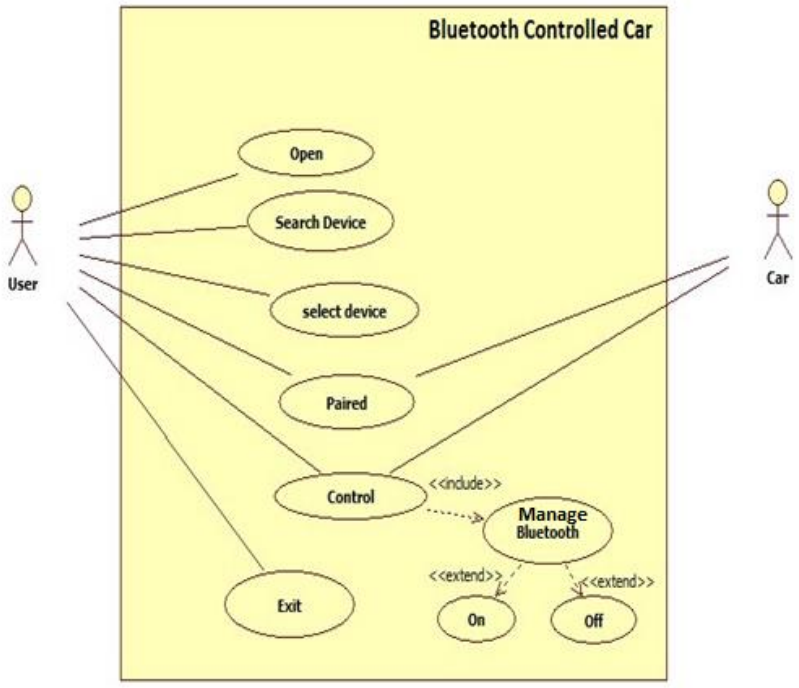



Figure 9: Use Case Diagram

### 4.2.2 Use case specification

- **Primary actor:**

  User using the Android application is the primary actor of this use case.

- **Pre-condition:**

  The user should connect the Android application to the robotic car is connected.

- **Main scenarios:**

There are number of main scenarios in our application.

- Firstly the user should open the application on his mobile.
- Then built your HC-06 Bluetooth module pair with your mobile.

- Password for pairing is “1234” or “0000”.
- Write name of the slave Bluetooth name in the text box and then tap connect button to pair Bluetooth module.
- After connection built user will be able to control and give direction to the robotic car.
- The user can exit his application or he can switch off robotic car circuit.

### 4.2.3 Sequence Diagram

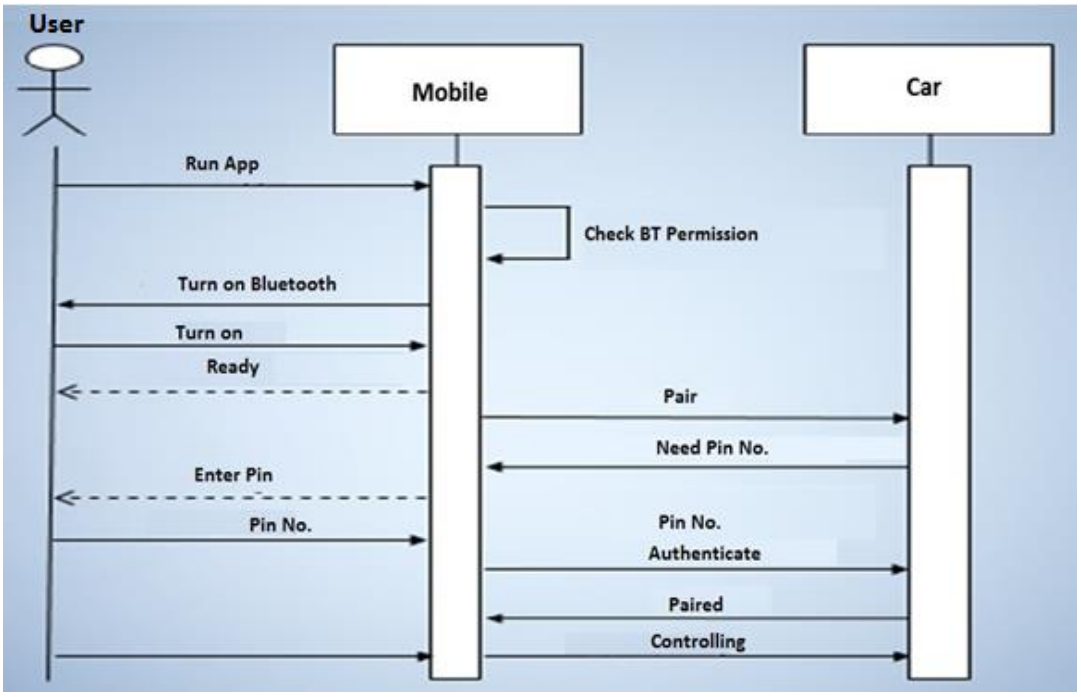



Figure 10: Sequence Diagram

Figure 10 illustrates When the application is started the availability of Bluetooth is checked. Before using this application, it should be kept in mind that both Android application and hardware system should have Bluetooth connection. If available the application prompts to scan the Bluetooth device which we write the name of Bluetooth in the text box in application. If not already connected, the application provides a dialog box giving a message of socket error and also if robotic car is not connected Bluetooth blinks, when it connected its light become stable. Robotic car must already be active when the application sends the request for connection.

It should be noted that for making pair with robotic car application will require pass key for more secure connection, after accepting valid pass key robotic car will be able to build a connection with application. User can now able to control car and moves it in all four direction. The user can now simply end the connection by closing the application or switch off the robotic car.

### 4.2.4 Class Diagram:

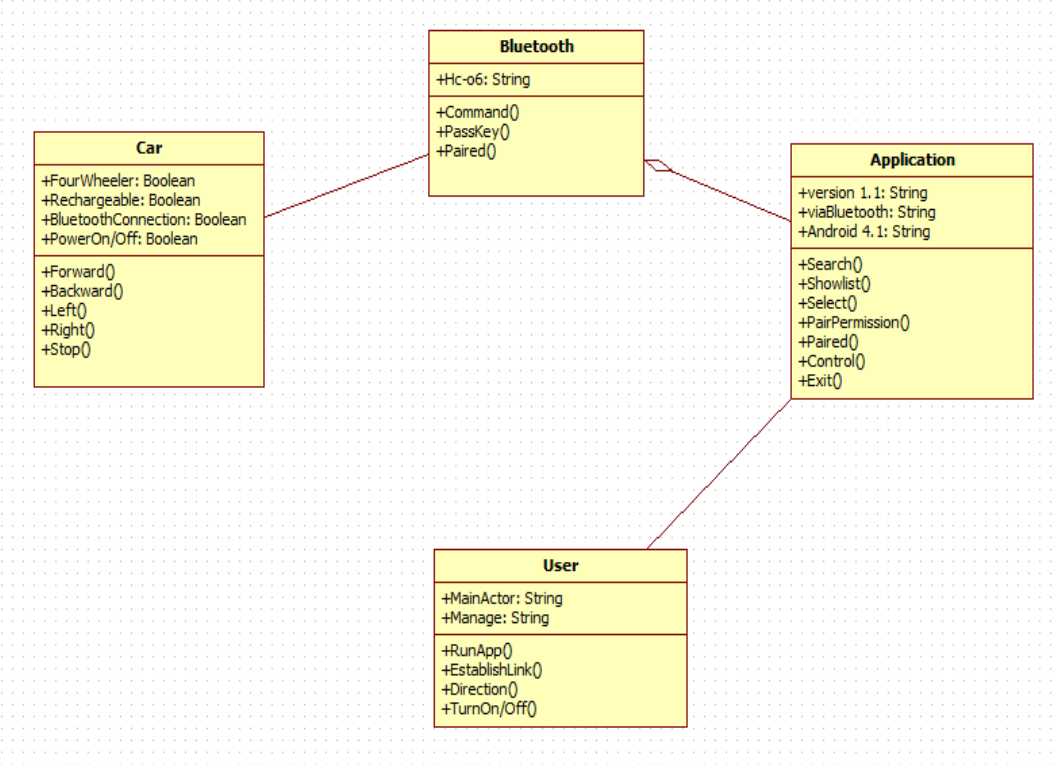



Figure 11: Class Diagram

### 4.2.5 Circuit Diagram

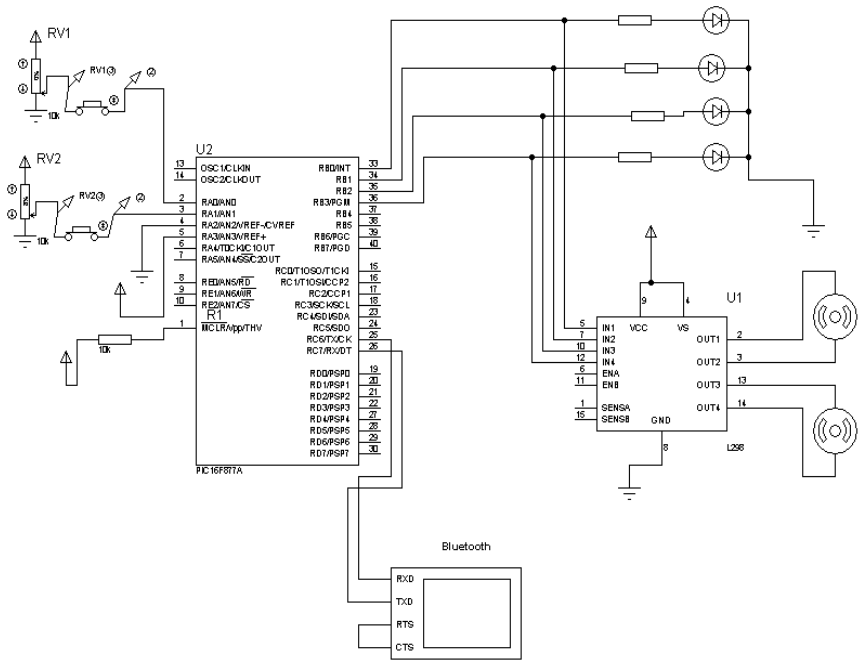



Figure 12: Circuit Diagram

# CHAPTER 5

# Implementation and technology

This chapter includes detailed information of the work during the implementation phase of the system which includes the design tools and technologies in detail. Best practice for coding for project is discussed in detail.

## 5.1 Tools & Technologies Used in Software Development

Tools used for development of our software are Eclipse, Android SDK, Proteus 7.8, Proton IDE and Blue stack as Emulator.

### 5.1.1 Eclipse

Eclipse is an open-source IDE donated to the community by International Business Machine (IBM). Java and the Android programs are written using Eclipse. Eclipse can also be used to program in other programming languages by means of external plugins. Also the plug-ins can extend the usability of Eclipse. Eclipse is developed by an open source community which has about 2000 open source projects related to different aspects of software development. Syntax-highlighting editor, incremental code compilation, a thread-aware source-level debugger, a class navigator, a file/project manager, and interfaces to standard source control systems are the features provided by Eclipse, once combined with JDT.

Eclipse was started as project for IBM which later in 2001 was released as an open source project. An association was then formed for further development of Eclipse as open source, and the original board members were Borland, IBM, Merant, QNX Software Systems, Rational Software, Red Hat, SusE, TogetherSoft and WebGain. Later in 2004 the Eclipse foundation was formed. It is a nonprofit organization responsible for governing the Eclipse projects. Figure 13 shows the basic window of the Eclipse.

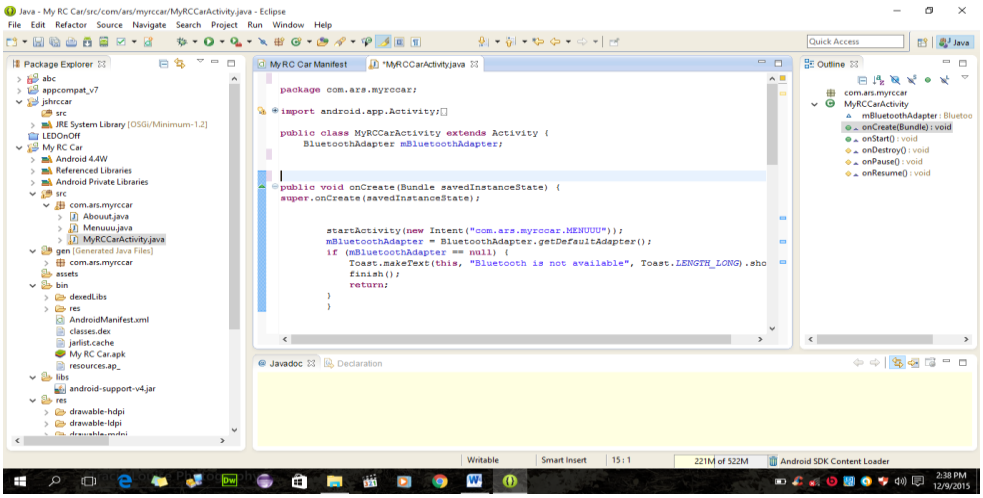


Figure 13: Eclipse user interface.

Above screenshot shows a simple user interface of Eclipse IDE. The whole figure is known as the work bench window. The window consists of a package explorer view, hierarchy view and Java outline view, wizards for creating the Java elements, Java editor, short cut bar, menu bar and a tool bar, as shown in figure 13. The Package explorer shows the entire project that is currently in the workspace and the project highlighted in the package explorer is the project that is currently in use. The Type hierarchy view shows the sub and super hierarchies whereas the Java outline view shows the classes and the Java units. Wizards for creating the Java elements consist of the Java classes, packages and interfaces. The Java editor is where all the codes for application development are written. The codes can be highlighted, edited or fixed in this area. All the commands such as edit, run, build and others are included in the shortcut bar, menu bar and tool bar. To develop the client-side application of the project in the Android platform using the Java programming language, Eclipse was used.

### 5.1.2 Android Software Development Kit and Environment

The Android SDK is a collection of software development kits used to develop applications in the Android platform. The Android SDK is available for free to download and use from the official Android developers' website. The SDK is available for all operating systems. It can be used alone to develop an application with a command prompt or can be used with IDEs such as Eclipse and NetBeans.

The IDE provides a graphical interface for easier and faster development of an application. The Android SDK consists of the following features:

- Required libraries
- Debugger
- An emulator
- Documentation for the Android application program interfaces (APIs)
- Sample source code
- Tutorials for the Android operating System.

For each release of a new version of Android, a corresponding SDK is released. A separate SDK is available for a separate version of Android. In order to use the latest feature in Android, developers need to install the latest SDK depending on the phone's Android version. To develop an application Android provides a custom plug-in for Eclipse called ADT. ADT provides an integrated environment for development of an application by extending the abilities of Eclipse. This helps developers to create an Android project, design an application user interface, debug an application and export and import an application package. The native language used for the Android application development is Java for which Java Development Kit (JDK) is needed. JDK can be downloaded for free from official Sun Microsystems website. All the above mentioned plug-ins and IDEs are available for free downloads and complete sets of instructions on installation are available on the Android official website.

### 5.1.3 Blues tack

The Blue stack App Player is designed to enable Android applications to run on Windows PCs and Macintosh computers. Blue stack Emulator is a desktop application that emulates an Android Phone. It provides a virtualized environment in which we can test. It also provides an isolated environment for application prototypes. Blue stack Emulator is designed to provide comparable performance to an actual device.

Table 5.1: System Requirements

| System Requirements | |
|---|---|
| Processor | 64 bit System: 2.13 GH or faster.<br>Recommended 2GH |
| RAM | 64 bit System: 4 GB or higher.<br>Recommended 4GB |
| Hard Disk | Minimum 4GB or higher<br>Recommended 4GB |
| Size | 262 MB |
| Operating System | Windows 10 |

### 5.1.4 Proton IDE:

Proton IDE is a professional and powerful visual integrated development environment (IDE) which has been designed specifically for the Proton Plus compiler, Proton IDE accelerates product development in a comfortable user environment without compromising performance, flexibility and control. Provide information about the device used, the amount of data used and the version number of the project and also date and time. The proton IDE enables you to install and start your preferred programming software from within the development environment. This enables you to compile and then program your microcontroller with just a few mouse clicks. The Proton IDE has been designed with flexibility in mind with support for IDE plugins.

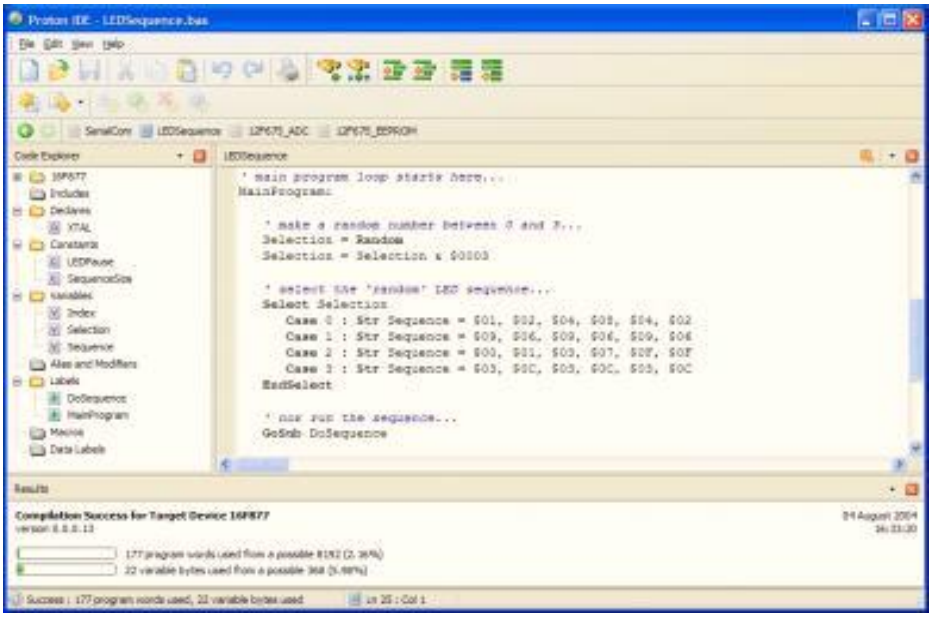


Figure 14: Main interface of Proton IDE

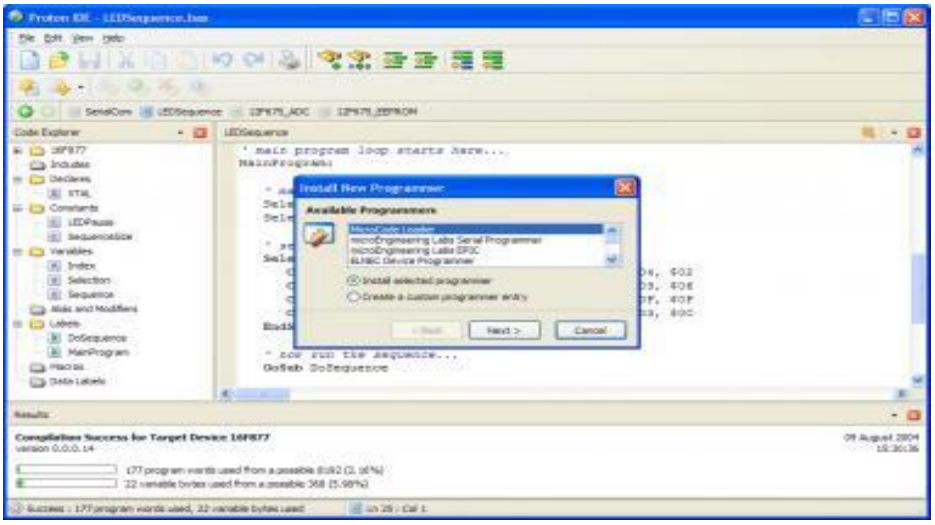


Figure 15: Select program to run on Proton IDE

### 5.1.5 Proteus 7.8

- Proteus is used to burn programming code in microcontroller.
- Proteus is very lenient in circuits designing and it works on ideal conditions like if you don't add pull up resistors etc. in the Proteus then it won't bother at all and runs the circuit. So, as the circuit designing is concerned you must have sound knowledge of components, only then you can design a working circuit

- **The main and foremost use of Proteus is the PCB designing**.

  In normal practice, I myself first design the circuit on either the wero board or then when I got sure that my circuit is fully working then I design its PCB in the Proteus.

- **Second use of Proteus is the Code Checking**.

  When you are designing some code then it's quite annoying to burn the microcontroller several times. Instead, design a circuit in your Proteus and after that simply design your PIC code and then run it on Proteus and get your best result and now when you are sure you are getting perfect output then burn you're PIC and test it on your hardware.

- **Click on Proteus ISIS** and then right click the image and open in new tab to get a better look of the image. In the central area, we design our circuit i.e. place the components and then join them.

## 5.2 Abstract:

This document consists of coding standards that are used while designing the software. Moreover, it contains a brief report of coding walkthrough and coding inspection. Coding walkthrough has been done by the people from outside of group member. However, coding inspection is been done by team members.

### 5.2.1 Coding Standards

#### 5.2.1.1 Coding standards used in development:

The following rules have been followed during the development phase of our project:

- Coding has been done as simple and readable as possible. Coding is done in a way that it is readable and understandable to any person who knows a little programming.
- No GOTO statement has been used to avoid complexity.
- Comments have been used for better understanding of the code. Where ever new functions and methods are coded their small description is mentioned.

- Meaningful comments have been used before classes and piece of code for documenting the code properly.
- Incomplete codes have been removed and the codes that are to be used in future are comment properly.
- Constants are not hard coded. They have been put in meaningful named variables.
- The indentation and Braces in the coding are symmetric and consistent.
- The variable names are according to their purpose.
- Empty white spaces have been used to enhance readability and are extremely helpful in testability and maintainability.
- Code file is well commented, it is very easy to understand.
- Code lines are not long to avoid difficulty to understand.

### 5.2.2 Debugging

Debugging is the process of finding and locating errors. Different techniques are used for debugging during the development of the project which is discussed below:

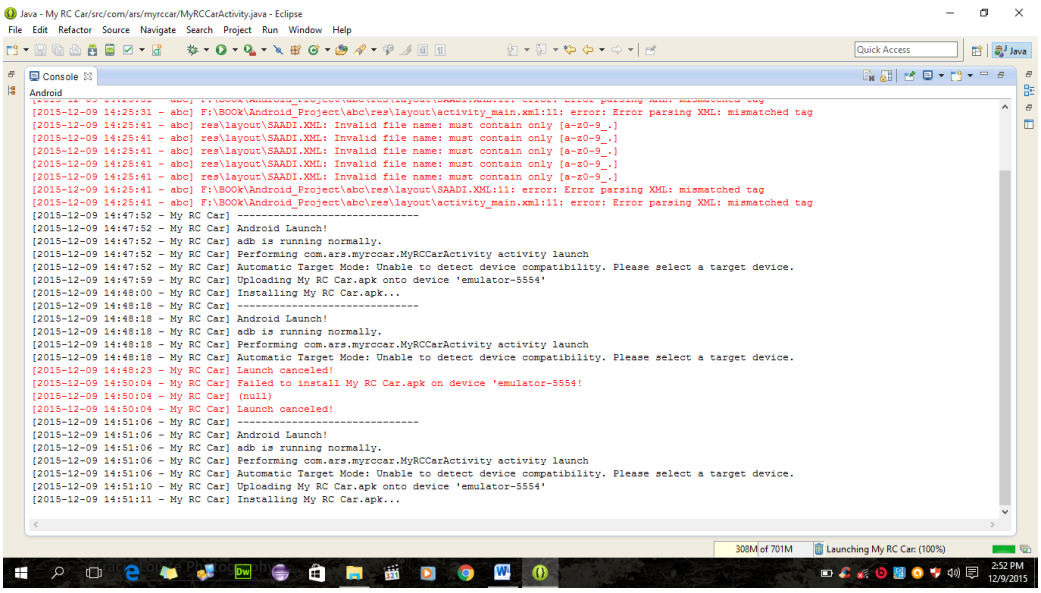



Figure 16: Eclipse Interface shows debugging

In this software application, debugging has been done by putting break points in the code. Android emulator and Eclipse provides the tools to debug which has been widely used for the debugging of the application. Android application

development also provides different other tools to debug. We have used eclipse debugging environment to debug our application.

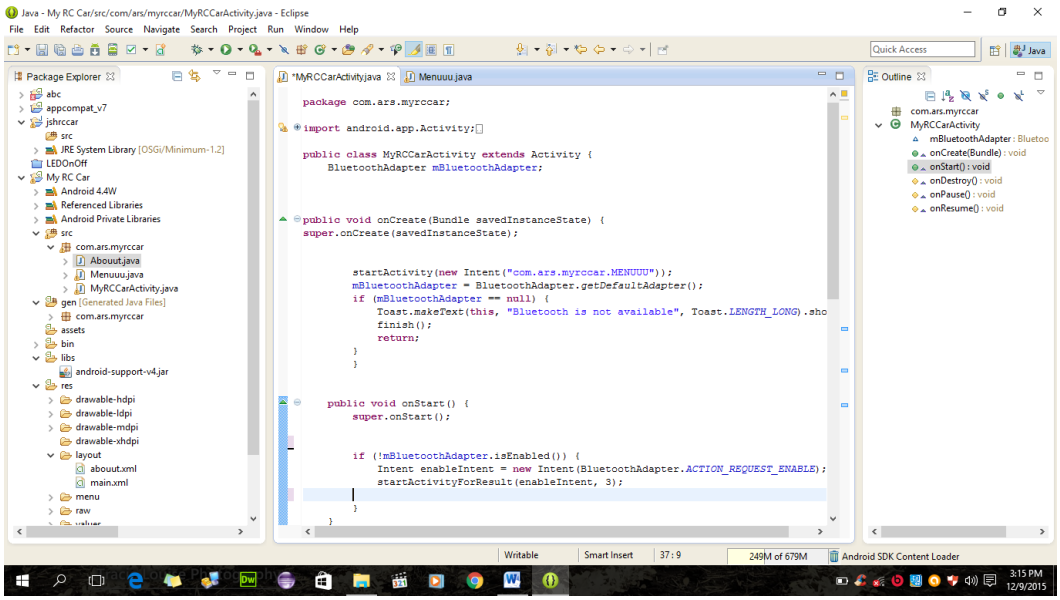



Figure 17: Eclipse Interface and Development Environment

Moreover the code has been reviewed by group members and the supervisor to avoid errors and misbehavior of the application.

### 5.2.3 Code walkthrough

A source code walkthrough often is called a technical code walkthrough or a peer code review. This is also known as dry run test. A typical walkthrough scenario can be of a developer inviting his technical lead to review prior to implementation. Often the modified code is represented by the comments in the code.

#### 5.2.3.1 Code Walkthrough Objectives:

The objectives of code walkthrough are given below

**Objective** - Check the code for:

- Standards coding violations
- Lack of clarity in coding
- Variable related to their mean
- Algorithmic and logical Errors
- Inconsistency

**Resources -** code walkthrough requires:

- Author
- Developer
- Designer
- External programmers
- Objects
- Code listing
- compiler listings
- Technical documentation.

### 5.2.3.2 Code walkthrough details

Table 5.2: Table 1 Code Walkthrough

| | |
|---|---|
| Source file names | AndroidManifest.xml<br>Main.xml<br>hardmenu.xml<br>About.xml<br>String.xml |
| Name of reviewer | Khalil Ahmad Sanghi |
| Reviewer Position | Student |
| Feedback | Approved |
| Dated | 18-Oct-2015 |

Table 5.3: Table 2 Code Walkthrough

| | |
|---|---|
| Source file names | About.xml<br>String.xml |
| Name of reviewer | Junaid Ahmad Butt |
| Reviewer Position | Student |
| Feedback | Approved |
| Dated | 20-Oct-2015 |

Table 5.4: Table 3 Code Walkthrough

| | |
|---|---|
| Source file names | mycarActivity.java<br>Menuuu.java |
| Name of reviewer | Haroon-ur-Rashid |
| Reviewer Position | Student |
| Feedback | Approved |
| Dated | 22-Oct-2015 |

Table 5.5: Table 4 Code Walkthrough

| | |
|---|---|
| Source file names | BuildConfig.java<br>R.java |
| Name of reviewer | Saddam Hussain |
| Reviewer Position | Student |
| Feedback | Approved |
| Dated | 25-Oct-2015 |

### 5.2.4 Code Inspection:

In code inspection code is viewed dynamically for errors. It is done after code is compiled. It requires an experienced staff that should be able to do it in systematic manner.

#### 5.2.4.1 Code Inspection Objectives:

**Objective -** Check the code for:

- Jumps into loops
- Nonterminating statements
- Use of uninitialized variables
- Incompatible assignments
- Array indices out of bounds
- Improper storage allocation and de-allocation
- Mismatches between actual and formal parameter in procedure calls
- Improper modification of loop variables

**Resources -** code inspection requires:

- Author of code
- Fellow Programmers
- Experienced leader
- Compiler
- Major Code inspection implementation

All code files have been reviewed dynamically. Each file has been shortly checked for the objectives checklist of code inspection. The code inspection was done by every team member.

#### 5.2.4.2 Code inspection details

Code inspection has been done by the team member. Each member inspects the other member's code files. The files has been exchanged among member and inspected.

Table 5.6: Table 1 Code Inspection

| | |
|---|---|
| Source file names | BuildConfig.java<br>R.java |
| Name of Inspector | Haroon-ur-Rashid |
| Position of Inspector | Student |
| Feedback | Approved |
| Dated | 28-Oct-2015 |

Table 5.7: Table 2 Code Inspection

| | |
|---|---|
| Source file names | AndroidManifest.xml<br>Main.xml<br>hardmenu.xml<br>About.xml<br>String.xml |
| Name of Inspector | Junaid Ahmad Butt |
| Position of Inspector | Student |
| Feedback | Approved |
| Dated | 29-Oct-2015 |

Table 5.8: Table 3 Code Inspection

| | |
|---|---|
| Source file names | mycarActivity.java<br>Menuuu.java |
| Name of Inspector | Saddam Hussain |
| Inspector Position | Student |
| Feedback | Approved |
| Dated | 29-Oct-2015 |

Table 5.8: Table 4 Code Inspection

| | |
|---|---|
| Source file names | Main.xml<br>hardmenu.xml<br>About.xml |
| Name of Inspector | Khalil Ahmad Sanghi |
| Inspector Position | Student |
| Feedback | Approved |
| Dated | 29-Oct-2015 |

# CHAPTER 6

# Implementation and technology

This chapter uncovers the different aspects of tests that are done on the software. Each and every test that has been documented has been explicitly categorized. Test cases for which test has been performed are explained in detail in appendix of document.

## 6.1 Introduction:

The product specified in this SRS is Android phone Application. This is an application which allows the user to use his application to remotely control robotic car via Bluetooth and moves device in all four direction.

## 6.2 Scope of Testing

The Scope of testing this application was to test whether the user can control his robotic car via Bluetooth without interruptions and data loss. The application is also been tested whether one or more user can control robotic car or not at a same time. The application is also tested for the range from which the robotic car can be accessed remotely.

On the Hardware side the whole circuit is tested whether it works fine, everything in the circuit is operating well and does not affect the performance of the hardware. The server is also tested for the range to which it can perform it works.

## 6.3 Test Plan Strategy

The primary objective of the testing was to check the efficiency and effectiveness of the application and to check the range to which the transmission between hardware and user application is smooth and uninterrupted.

**To test the application following techniques has been used:**

### 6.3.1 White Box Testing

In white box testing the internal structure of the system is considered.

There are 3 types of white box testing

- Statement Coverage: Every statement in project program is executed at least once.
- Branch Coverage: For branch coverage all the branches in our project code is tested thoroughly.
- Path Coverage: For path coverage all linear independent paths are executed and tested.

**Methodology:**

- White box testing was performed by all participants.
- Statement Coverage testing was performed.

**Participants:**

- Haroon-ur Rashid
- Saddam Hussain
- Junaid Ahmad Butt

### 6.3.2 BLACK BOX TESTING

Black box testing for this project is done and all the functionalities which were explained in the SRS are been tested and all the tests are successful. The project is fully working and it is performing all the required functionalities.

#### 6.3.2.1 Unit Testing

Individual tests for the every module have been under taken. The modules that were tested are:

- Transmission
- Connectivity
- Device
- Range

**Participants:**

- Junaid Ahmad Butt
- Saddam Hussain
- Haroon-ur-Rashid

**Methodology:**

- All project participants have performed testing on each module of the software.
- The transmission code has been tested by an external supervisor and he didn't write the code for transmission between the robotic car and the user application.

#### 6.3.2.2 Integration Testing

The Whole System has been tested after the integration of modules by using the following approaches and techniques:

**Big Bang Approach:**

The entire software and hardware package has been tested as a whole.

**Incremental construction strategy:**

- During the development of the software small tests have been performed while incrementing the functionalities and modules.
- By using the incremental strategy the errors were comparatively easy to find and were fixed.

**Participants:**

- Saddam Hussain
- Haroon –ur-Rashid
- Junaid Ahmad Butt

**Methodology:**

- Different units or parts were united in the main program.

- Also different parts of hardware were united.
- Incremental Construction strategy was used.
- All members of group did this job.
- The interface and hardware components were tested to find the errors.
- Each error was handled accordingly.

#### 6.3.2.3 System Testing

Software was validated by combining different elements of system to check whether they give the desired output or not. Both hardware and user applications were tested at the same time and the connection and transmission between the hardware and user application was successful and reliable.

**Participants:**

- Engr. Zaman Fakhar
- Haroon-ur-Rashid
- Junaid Ahmad Butt
- Saddam Hussain

**Methodology:**

System testing was performed by all participants.

#### 6.3.2.4 Acceptance testing:

The requirements mentioned in the SRS were checked by the testing the software and the software are meeting all the requirements**.**

Internal Acceptance Testing (Also known as Alpha Testing) was done by the project supervisor and it was concluded that the software meets all the desired requirements.

**Participants:**

Engineer Zaman Fakhar

## 6.4 Requirements Verification and Validation

The Purpose of Requirements Verification and validation was to check the software system that it meets its requirements specification and fulfills its intended purpose.

### 6.4.1 Requirements Verification:

The whole System is entirely tested and verified by imposing different conditions on different phases of the development.

### 6.4.2 Requirements Validation:

The Entire system has been developed, tested thoroughly and validated. It performs all the required functionalities as described in the software requirement specification document.

## 6.5 Solution validation and Analysis of data

### 6.5.1 Solution Validation:

We worked on establishing wireless serial communication with a four wheeler car using any Bluetooth device. We worked with both computer and mobile Bluetooth and we become successful after lots of trial and error. Its operation is very safe and there is almost no risk in handling this process. The total cost is also much economic. It can be used in various sectors in our daily life easily by adding extensions.

### 6.5.2 Analysis of the Data:

Our model car is tested by us many times. We have found a very positive output from it. Thought at first we are facing problem setting the baud rate up to 9600 for microcontroller. But now both the mechanical and electrical parts are working well with each other. The range of controlling the car from any smartphone up to 15 meter but it is quite satisfactory. The operating application which was made by us is very simple to use so everyone is controlling the car by it. All the buttons controls functions of the android application works perfectly while controlling robotic car.

## 6.6 Test Environment and testing Schedule

The Android phone application has been tested in Eclipse and Blue Stack emulator.

Table 6.1: Testing Schedule

| Testing Activities | Start | End | Person Responsible |
|---|---|---|---|
| Designing Test Cases | 08th October 2015 | 12th October 2015 | Saddam Hussain |
| Categorize Test Cases | 15th October 2015 | 20th October 2015 | Haroon-ur- Rashid |
| White Box Testing | 22th October 2015 | 23th October 2015 | Junaid Ahmed Butt |
| Unit Testing | 24th October 2015 | 25th October 2015 | Haroon Rashid |
| Integration Testing | 25th October 2015 | 27th October 2015 | Junaid Ahmed Butt Sadddam Hussain |
| System Testing | 27th October 2015 | 28th October 2015 | Haroon Rashid, Junaid Ahmed Butt and Sadddam Hussain |
| Acceptance Testing | 05th November 2015 | 11th November 2015 | Engr. Zaman Fakhar |

## 6.7 Resources

### 6.7.1 System Model:

It is the motherboard developed by Intel Cooperation in June 2011. Its properties are shown below:

- **System Model:** Hp Pavilion
- **Max Memory Size:** 8 GB
- **Memory Types:** DDR2

### 6.7.2 Processor:

- **Model:** Intel Core i5 n Series
- **Number of Cores:** 4
- **Number of Threads:** 4
- **Clock Speed:** 2 GHz
- **L2 Cache:** 4 MB

### 6.7.3 RAM:

Kingston DDR2 800 MHz 4GB x1

### 6.7.4 Hard drive:

- **Size:** 720 GB
- **Cache:** 4 MB
- **Interface:** SATA 6 Gb/s
- **Spin Speed:** 5600 rpm
- **Average Latency:** 4.16ms
- **Annual Failure Rate:** 0.34%

### 6.7.5 Software Environment

- Eclipse Juno
- Android SDK
- Java Development Kit

## 6.8 Test Cases Implementation

Designed Test cases are in appendix with full description

### 6.8.1 White Box Testing

Table 6.2: White Box Testing

| Test Engineer:<br>Junaid Ahmed Butt | Reviewed:<br>Haroon Rashid<br>and<br>Saddam Hussain | TestDate:09,Nov 2015 |
|---|---|---|
| **WHITE BOX TEST RESULT** | | |
| Files for which branch coverage test was not performed do not contain branches | | |
| **Code File Name** | **Statement Coverage Test** | **Branch Coverage** |
| **mycarActivity.java** | Successful | Successful |
| **Menuuu.java** | Successful | Successful |
| **BuildConfig.java** | Successful | Successful |
| **R.java** | Successful | Successful |

### 6.8.2 Black Box testing

Table 6.3: Black Box Testing

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-02 | **QA Test Engineer** | Haroon-ur-Rashid |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Saddam Hussain |
| **Test Date:** | 25th November 2015 | | |
| **Revision History** | None | | |
| **Objective** | Controlling device | | |
| **Pre-Condition** | Devices connected, access allowed. | | |
| **Environment:** | **Windows 10 an android phone 4.2** | | |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Controlling device | Device control |
| **Comments** **The test was successful.** | | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* | |

### 6.8.3 Unit Testing

Table 6.3: Unit Testing 1

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-03 | **QA Test Engineer** | Haroon-ur- Rashid |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Saddam Hussain |
| **Test Date:** | 17th November 2015 | | |
| **Revision History** | None | | |
| **Objective** | Start application | | |
| **Pre-Condition** | JDK not Installed | | |
| **Environment:** | **Windows 8.1** | | |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Install JDK 1.7 or above | Installed |
| 2 | Start application | Application Running |

| | |
|---|---|
| **Comments**<br>**The test was successful.** | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* |

Table 6.4: Unit Testing 2

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-04 | **QA Test Engineer** | Haroon-ur- Rashid |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Saddam Hussain |
| **Test Date:** | 21th November 2015 | | |

| | |
|---|---|
| **Revision History** | None |
| **Objective** | Power on/ Start Hardware device |
| **Pre-Condition** | Battery not connected |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Check Hardware circuit | Circuit is working |
| 2 | Start Device | Device running |

| | |
|---|---|
| **Comments**<br>**The test was successful.** | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* |

Table 6.5: Unit Testing 3

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-05 | **QA Test Engineer** | Haroon-ur- Rashid |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Saddam Hussain |
| **Test Date:** | 17th November 2015 | | |
| **Revision History** | None | | |
| **Objective** | Start application | | |
| **Pre-Condition** | JDK not Installed | | |
| **Environment:** | **Windows 8.1** | | |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Install JDK 1.7 or above | Installed |
| 2 | Start application | Application Running |

| | |
|---|---|
| **Comments**<br>**The test was successful.** | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* |

Table 6.6: Unit Testing 4

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-06 | **QA Test Engineer** | Saddam Hussain |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Haroon-ur-Rashid |
| **Test Date:** | 22th November 2015 | | |
| **Revision History** | None | | |
| **Objective** | Connect to Hardware device | | |
| **Pre-Condition** | Both Hardware and Application connected to same network | | |
| **Environment:** | **Windows 10 and Android Phone 4.2** | | |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Power on Hardware | Hardware Running |
| 2 | Start Android Application | Application running |
| 3 | Enter Pass Key and start controlling | Passkey matched, Devices Connected. |

| | |
|---|---|
| **Comments**<br>**The test was successful.** | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* |

Table 6.7: Unit Testing 5

| | | | |
|---|---|---|---|
| **Test Case Reference ID:** | TC-07 | **QA Test Engineer** | Saddam Hussain |
| **Test case Version:** | 1.0 | **Reviewed By** | Junaid Ahmed Butt and Haroon-ur- Rashid |
| **Test Date:** | 25th November 2015 | | |
| **Revision History** | None | | |
| **Objective** | Directing Car | | |
| **Pre-Condition** | Devices connected | | |
| **Environment:** | **Windows 10 and Android Phone 4.2** | | |

| **Step No.** | Execution description | **Procedure result** |
|---|---|---|
| 1 | Enter Passkey | Passkey matched |
| 2 | Controlling car | Access allowed to control |

| | |
|---|---|
| **Comments**<br>**The test was successful.** | |
| | ☒*Passed* ☐*Failed* ☐*Not Executed* |

# CHAPTER 7

## Interface Design

The first screen of our application shows the control panel which contain Control, Help and About buttons. User can choose their desire option.




Figure 18: Application Control panel

When tap to the control button in control panel user will enter into the main interface of the application. User will be able to control car in different directions.

- When press “up arrow” the robotic car moves FORWORD.
- When press “DOWN ARROW” the robotic car moves REVERSE.
- When press “LEFT ARROW” the robotic car turns LEFT.
- When press “RIGHT ARROW” the robotic car turns RIGHT.

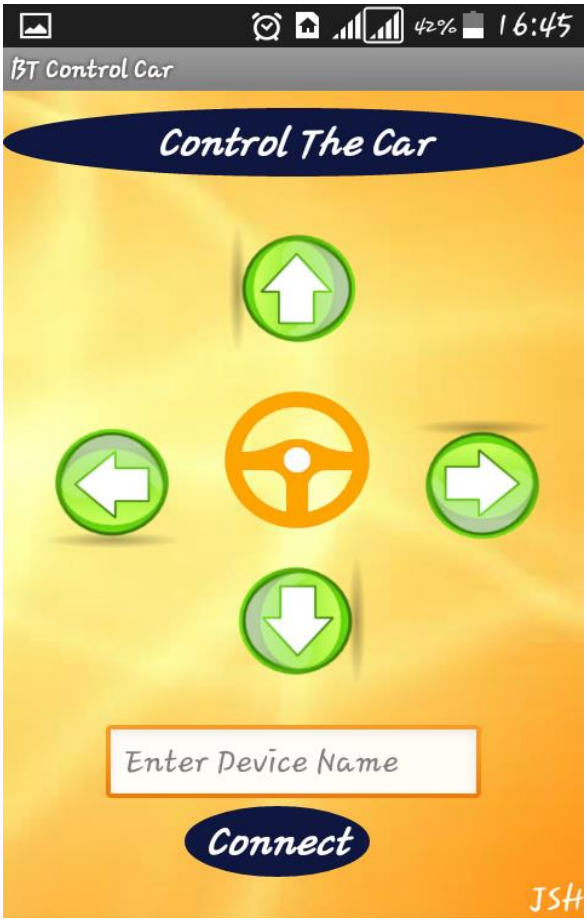



Figure 19: Main controlling interfaces

If a beginner does not know how to control car, application have help option so that user can able to knowing how to operate application.

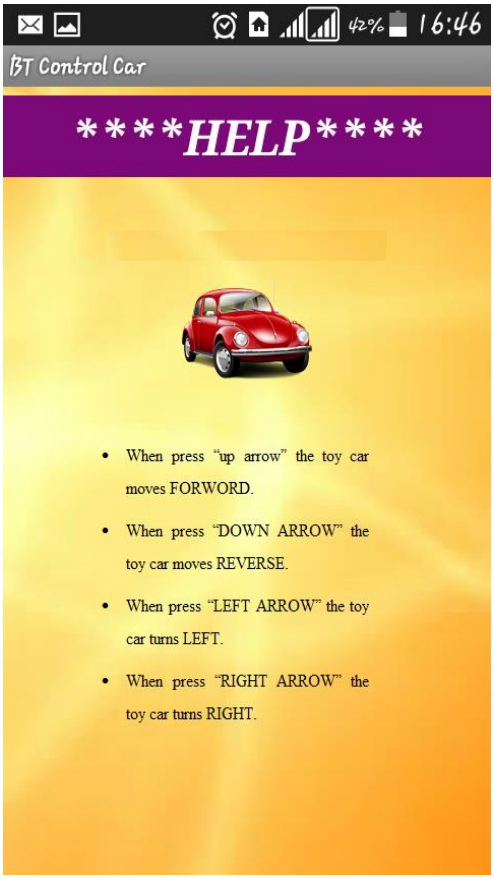



Figure 15: Help interface

User can also see information about all developers of the application.




Figure 16: About interface.

## CONCLUSIONS & RECOMMENDATIONS

### Results:

The result which we got while doing this project is very encouraging. It, at first receives command from the application by Bluetooth module of the car and performs the commands instantly. The positive side of this project can be summed up as bellow:

- Bluetooth control mode gives the operator freedom to move it anywhere within its range.
- The android application which was made is so simple that it can be installed in any android supported smart phone. So anyone can run it if he or she has the application.
- This wireless control vehicle can be useful in our daily day to day jobs by adding some extension to it. For example if a robotic arm is attached to it can perform as an assistant to worker.

## Discussion:

A few years ago developing application was meant only for PCs. Hardware and software requirements for PC applications to be able to run are very high. Furthermore applications development requires dedication and involvement of a large number of skilled people. However with the launch of the smartphone the concept has changed. Now applications are also meant for mobile phones. To make mobile applications dedication and involvement of a large number of skilled people is not necessarily required. Such applications can be made by a single person and an application developer can get involved in such development as a profession or as a hobby.

- Android has a good and well conducted market for developers to sell their applications. These types of platform have given people chances to expose their skills. Moreover Android being an open source people gets more interested in application development. Open source means that the source code of the applications are available to public for use and modification from its original design. Android has got plenty of guides on setting up the environment for application development, getting stated with the application development and putting developed applications in the market for selling. So applications can be developed easily if developers' guides are followed properly. Also several sensors are available on the Android phones. As this project used Bluetooth, it was found out that Bluetooth can be used for different purposes other than using it as a communicator for a remote control for computers.
- We faced some problems while setting the baud rate for microcontroller. Problems arise when we try to work with higher baud rate (>2400) with the internal 1MHz clock of microcontroller. Because according to Datasheet the error is too high (almost 7%). Moreover internal clock gives a default error of +/-10%. So we had to use a 4MHz crystal as external clock source. The 4MHz clock speed and 9600 baud rate gives significantly low error (<0.2%). For adding the crystal we had to write the fuse bits of microcontroller.
- Bluetooth is a good communication system while working in a short range (<30m). But while working in a longer range it is not a wise decision.

- We use H-Bridge motor driver. That drives motor into different directions (differential drive) according to given commands.
- Another limitation of Bluetooth Technology is it is much power consuming and drains the battery quickly.
- While powering the motor and the microcontroller from the same source some problem s were faced due to the noise. We used two different powers for motors and microcontroller for safety.

## Conclusions

We have successfully run our project. It can be used in various fields to make our life swifter and easy. Some possible and future implementations can be named as bellow:

- In house hold for carrying various things like trays of food or in other entertainment appliances like a moving sound system.
- In hospital or in other places we can implement it in wheelchair which can help the patient to operate easily.
- In office for carrying large amount of files from one place to another we can use this kind of arrangement.
- In shopping mall it can be used by the customer to carry their things from one place to another by smart trolley.
- Sometimes we face some situations where we cannot send any big things. In that case too we can use it as a model savior kits.
- Most importantly it can be transformed into a very good toy for the new generation kids who are now growing with a smartphone in his/her hand.

## Recommendations

Though our project is complete in our sense but it can be improved in many ways which we can't show for time or for various other causes like lack of resources etc. Following improvement of our project may be considered:

- **Voice recognition** can be added to the car such that it can be motioned by speaking directly through mouth.
- **Speed control** while using accelerometer can also be introduced.

- **High weight carrying properties** can be given to this car so that many more mechanical or other functions can be installed in it.
- **Friendly design** can be given to it so that it can be representing in more commercial way.
- **More functions** can be added to android application.
- **Distance of control** can be increased.

## References:

**http:// http://developer.android.com/**

**http://www.tutorialspoint.com/android/**

**https://github.com**

**http://stackoverflow.com/**

**http://www.androidhive.info/**

**http://www.circuitstoday.com/avr-microcontroller-tutorial**

**http://developer.android.com/guide/topics/connectivity/bluetooth.html**

# ABBREVIATIONS

**APP:** Application

**RF:** Radio Frequency

**IDE:** Integrated Development Environment

**JDK:** Stands for Java Development Kit

**GUI:** Graphical User Interface

**SDK:** Stands for Software Development Kit

**PCB:** Printed Circuit Board

**IBM:** International Business Machine

**ADT:** Android Development Tool

**API:** Application Programing Interphase

**RAM:** Random Access Memory

**ADT:** Android Development tool

**XML:** Extensible Markup Language